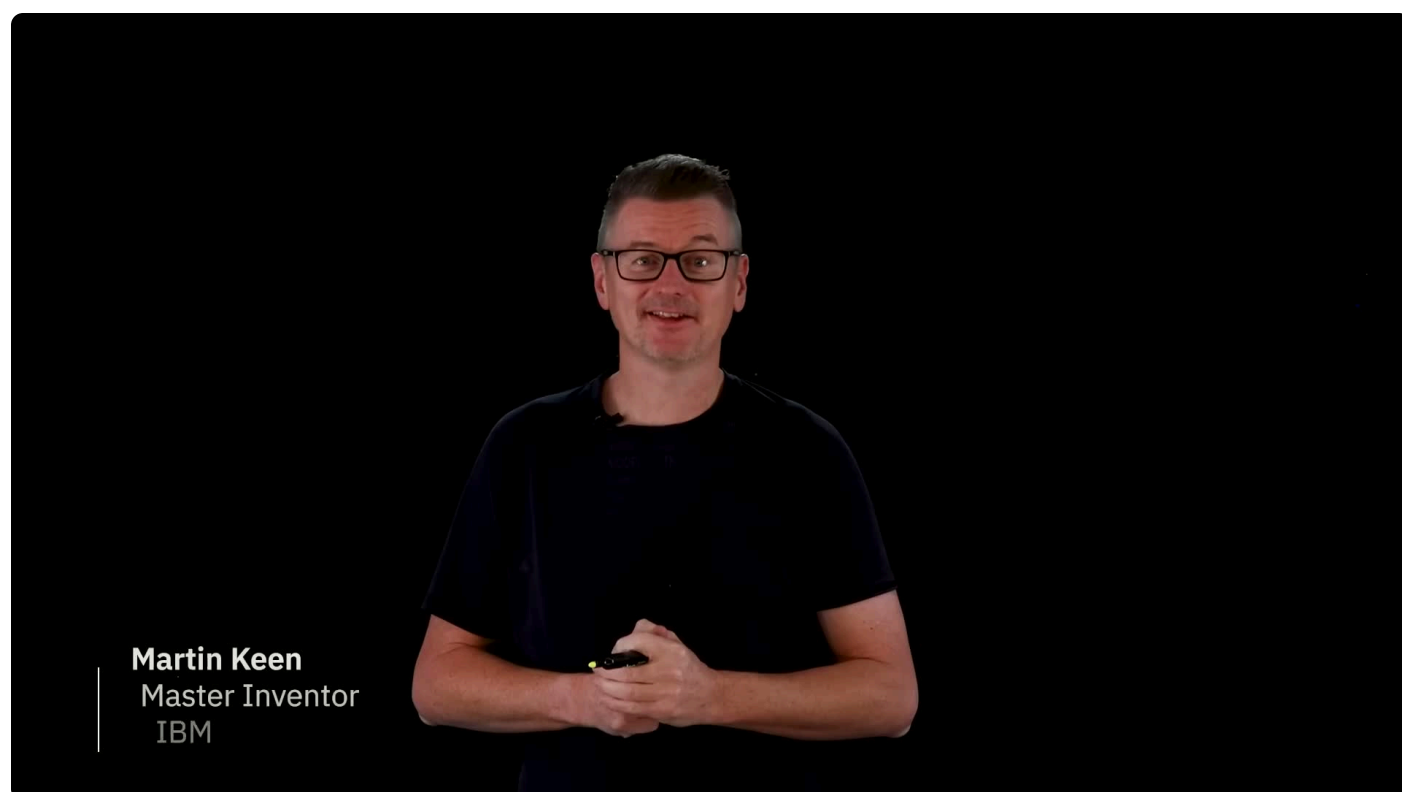


[← Submit another video](#)

RAG vs. Long Context: Choosing the Right Approach for LLMs

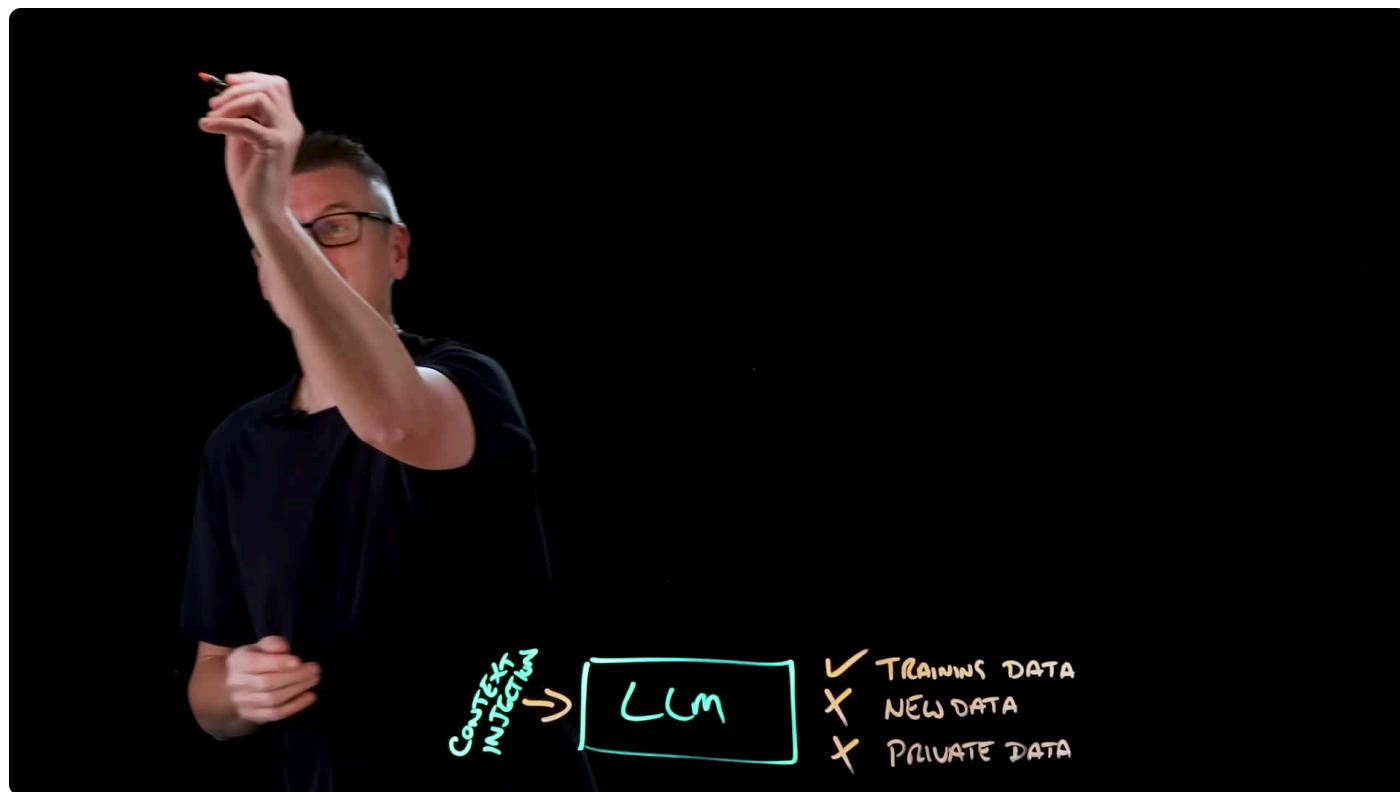
Generated by VidScribe

The LLM's Knowledge Limitation and the Need for Context Injection



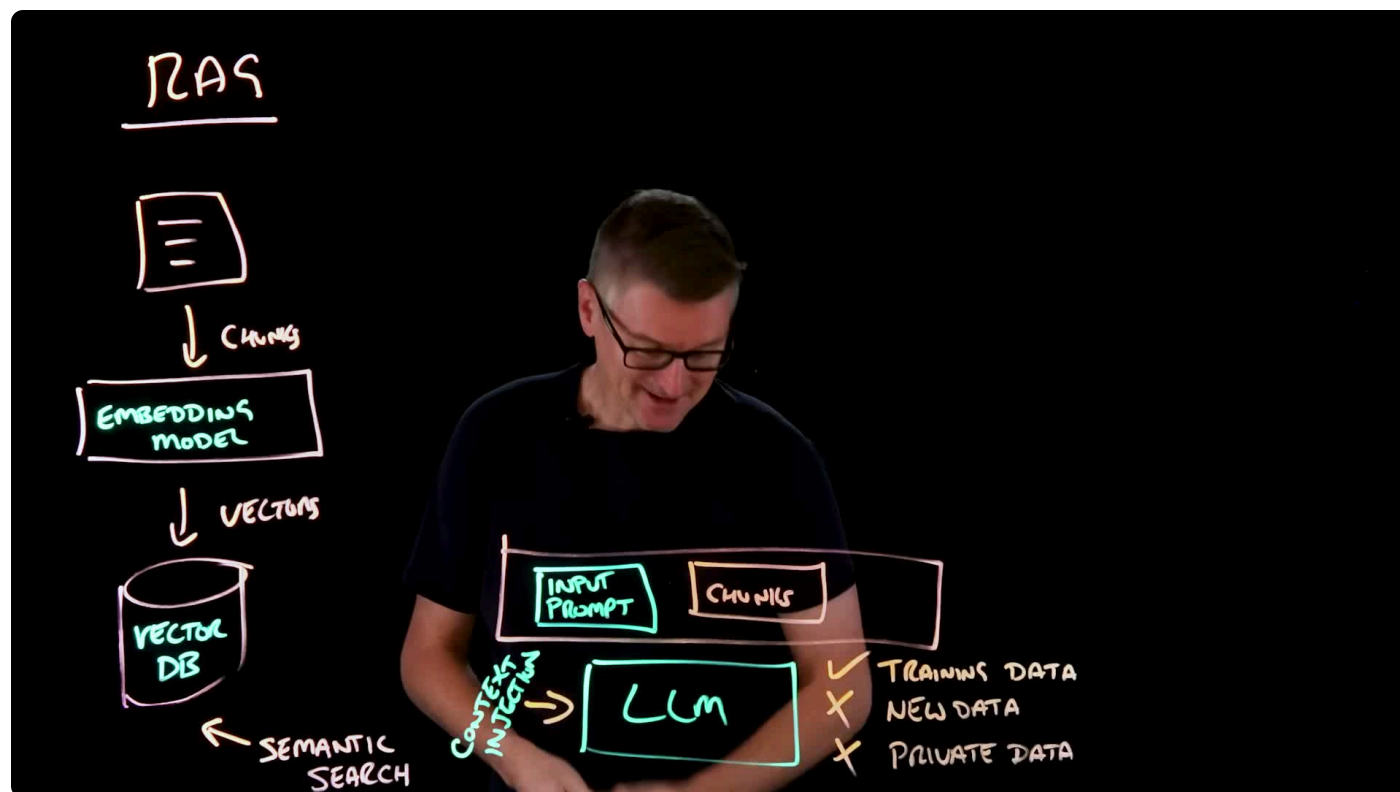
Large Language Models (LLMs) have a knowledge cutoff based on their training data and lack awareness of real-time events or private, proprietary information. To enable LLMs to access and utilize such data, the challenge of context injection must be addressed: how to effectively provide the right data to the model at the appropriate time.

Retrieval Augmented Generation (RAG)



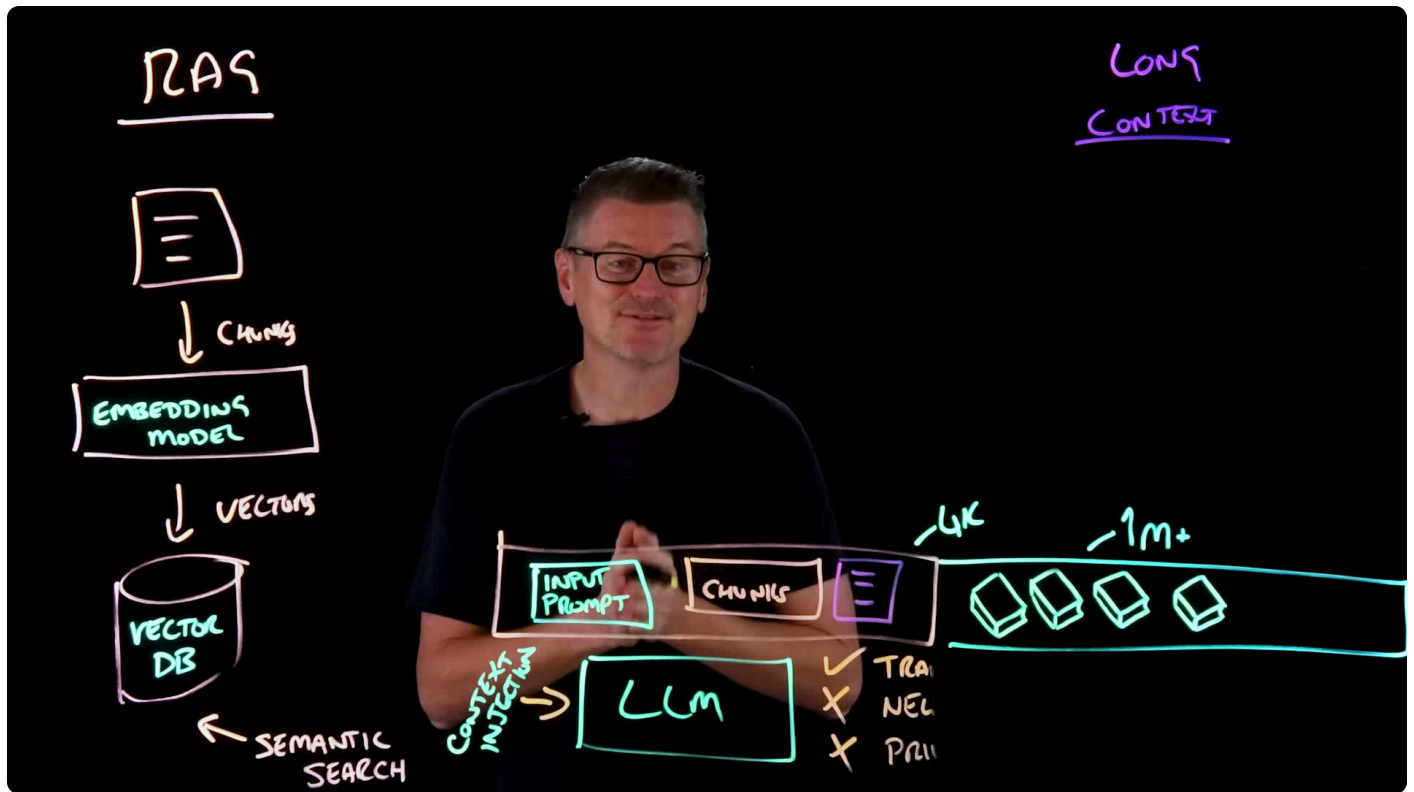
RAG is an engineering approach to context injection. It involves chunking documents (like PDFs, code, or books) into smaller pieces, converting these chunks into vectors using an embedding model, and storing them in a vector database. When a user asks a question, RAG performs a semantic search to find the most relevant chunks and injects them into the LLM's context window along with the user's prompt. However, RAG's effectiveness depends on the accuracy of its retrieval logic.

Long Context: The Model-Native Solution



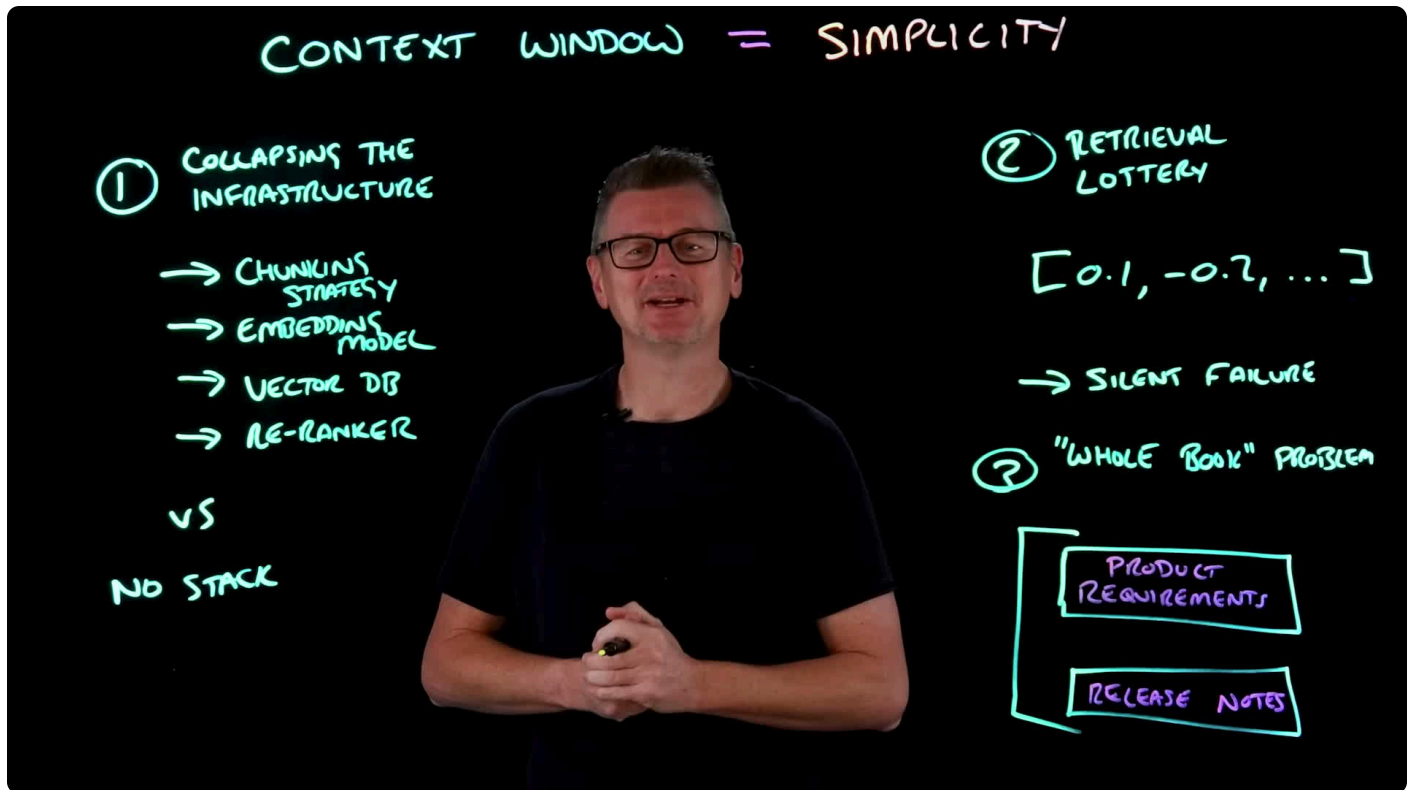
Long context is a more direct, brute-force method where documents are placed directly into the LLM's context window, bypassing embedding models and vector databases. The model's attention mechanism then handles the task of finding the relevant information. Historically, this was not feasible due to small context window sizes in early LLMs (around 4K tokens). However, modern LLMs boast significantly larger context windows (e.g., a million tokens or more), capable of holding vast amounts of text, prompting a re-evaluation of RAG's necessity.

Advantages of Long Context



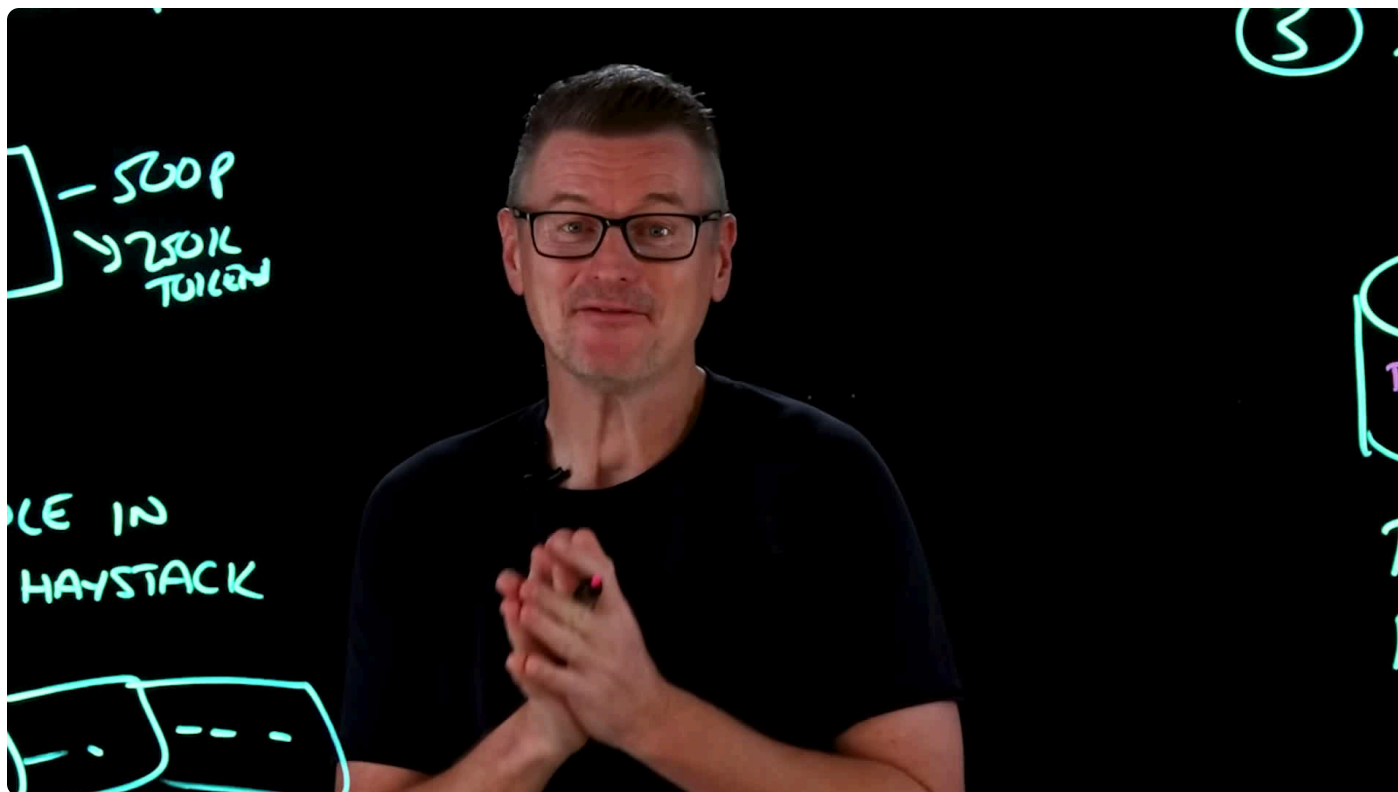
The shift towards larger context windows makes the long context approach attractive due to its simplicity. Three key advantages are highlighted: collapsing infrastructure by removing the need for chunking strategies, embedding models, vector databases, and rerankers; mitigating the 'retrieval lottery' by eliminating the probabilistic nature of semantic search and the risk of silent failures where relevant data is missed; and addressing the 'whole book problem' by allowing the model to access and compare entire documents, enabling it to identify gaps or omitted information that isolated snippets retrieved by RAG might miss.

Advantages of RAG



Despite the advantages of long context, RAG remains relevant for several reasons. First, long context incurs significant compute inefficiency, requiring the model to re-process large documents for every query, whereas RAG processes data once at indexing time. Second, RAG addresses the 'needle in the haystack' problem; while large context windows allow more data, the model's attention can become diluted, leading to missed information. RAG, by retrieving only the most relevant chunks, provides focused data. Third, for enterprise-scale data, which can be terabytes or petabytes, context windows are still insufficient, necessitating a retrieval layer like RAG to filter information.

Conclusion: Choosing the Right Approach



The choice between RAG and long context depends on the specific use case. For problems involving bounded datasets that require complex reasoning across entire documents, such as legal contracts or book summarization, long context offers simplicity and improved reasoning. However, for navigating the vast, 'infinite' datasets typical of enterprise knowledge, the vector database employed by RAG remains the most viable solution for data warehousing and retrieval.